

Hadoop WordCount Practical (SPPU LP2/DSBDA Style)

This is one of the most common Hadoop practicals. Even if you've never used Hadoop, for exam viva you mainly need to know:

1. What Hadoop is
 2. What MapReduce does
 3. Commands to run WordCount
 4. Mapper + Reducer logic
-

What is Hadoop?

Apache Hadoop is a framework used for:

- Storing big data
- Processing big data

It uses:

- HDFS → storage
 - MapReduce → processing
-

What is MapReduce?

Programming model for parallel processing.

It has 2 phases:

Phase	Work
Map	Convert input into key-value pairs
Reduce	Combine/count values

WordCount Example

Input:

```
hello world
hello hadoop
Mapper Output:
```

```
hello 1
world 1
hello 1
hadoop 1
Reducer Output:
```

```
hello 2
world 1
hadoop 1
```

Hadoop Modes

Mode	Meaning
Standalone/Local	Runs on single machine
Pseudo-distributed	Simulates cluster
Fully distributed	Multiple systems

Your practical usually uses:

Local Standalone Mode

Basic Hadoop Workflow on Ubuntu

Step 1: Open Ubuntu Terminal

Step 2: Check Java

Command

```
java -version
```

Step 3: Check Hadoop

Command

```
hadoop version
```

Step 4: Create Input File

Command

```
nano input.txt
```

Write:

```
hello world
```

```
hello hadoop
```

Save using:

```
CTRL + O
```

```
ENTER
```

```
CTRL + X
```

Java WordCount Code

Create file:

```
WordCount.java
```

Complete Java Code

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {
```

// Mapper Class

```
public static class TokenizerMapper
    extends Mapper<Object, Text, Text, IntWritable> {

    private final static IntWritable one = new IntWritable(1);

    private Text word = new Text();

    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString();

        String[] words = line.split(" ");

        for (String w : words) {

            word.set(w);

            context.write(word, one);
        }
    }
}
```

// Reducer Class

```
public static class IntSumReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values,
        Context context)
        throws IOException, InterruptedException {

        int sum = 0;

        for (IntWritable val : values) {

            sum += val.get();
        }

        context.write(key, new IntWritable(sum));
    }
}
```

// Main Method

```
public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(conf, "word count");

    job.setJarByClass(WordCount.class);
```

```
job.setMapperClass(TokenizerMapper.class);

job.setReducerClass(IntSumReducer.class);

job.setOutputKeyClass(Text.class);

job.setOutputValueClass(IntWritable.class);

FileInputFormat.addInputPath(job, new Path(args[0]));

FileOutputFormat.setOutputPath(job, new Path(args[1]));

System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

Explanation of Important Commands

Mapper

Processes input data.

Produces:

(key, value)
pairs.

Example:

hello → 1

Reducer

Combines values for same key.

Example:

hello → 1+1 = 2

IntWritable

Hadoop integer datatype.

Text

Hadoop string datatype.

context.write()

Outputs key-value pair.

line.split(" ")

Splits sentence into words.

Compile Hadoop Program

Step 1: Find Hadoop Classpath

Command

```
hadoop classpath
```

Step 2: Compile

Command

```
javac -classpath `hadoop classpath` -d wordcount_classes WordCount.java
```

Explanation

Part	Meaning
javac	Java compiler
-classpath	Hadoop libraries
-d	Output folder

Create JAR File

Command

```
jar -cvf wordcount.jar -C wordcount_classes/ .
```

Explanation

Creates executable JAR.

Run Hadoop Program

Command

```
hadoop jar wordcount.jar WordCount input.txt output
```

Explanation

Part	Meaning
hadoop jar	Run Hadoop application
wordcount.jar	JAR file
WordCount	Main class
input.txt	Input file
output	Output folder

View Output

Command

```
cat output/part-r-00000
```

Expected Output

```
hadoop 1  
hello 2  
world 1
```

Complete Flow Diagram

```
Input File  
  ↓  
Mapper  
  ↓  
(key,value) pairs  
  ↓  
Shuffle & Sort  
  ↓  
Reducer  
  ↓  
Final Output
```

Important Theory

What is HDFS?

Hadoop Distributed File System.

Used for:

- Distributed storage
-

What is Big Data?

Very large datasets difficult to process traditionally.

Features of Hadoop

- Scalable
 - Fault tolerant
 - Distributed
 - Open source
-

What is Key-Value Pair?

Example:

hello → 1

What is Shuffle and Sort?

Intermediate phase:

- Groups same keys together before reducer.
-

Important Viva Questions

Basic Questions

What is Hadoop?

Framework for distributed storage and processing.

What is MapReduce?

Programming model for parallel data processing.

What is Mapper?

Processes input and generates key-value pairs.

What is Reducer?

Combines values of same key.

What is HDFS?

Distributed storage system in Hadoop.

Intermediate Questions

Why IntWritable used instead of int?

Hadoop uses its own datatypes.

What does context.write() do?

Writes output key-value pair.

Why split() used?

To separate words.

What is local standalone mode?

Runs Hadoop on single machine without distributed setup.

Advanced Questions

What happens between Mapper and Reducer?

Shuffle and sort phase.

Why Hadoop scalable?

Can add more nodes easily.

Difference between HDFS and MapReduce?

HDFS	MapReduce
Storage	Processing

Why WordCount popular?

Simple example to understand MapReduce.

External May Ask Modifications

Convert Words to Lowercase

Inside mapper:

```
line = line.toLowerCase();
```

Remove Special Characters

```
line = line.replaceAll("[^a-zA-Z ]", "");
```

Count Character Frequency Instead

Split characters instead of words.

Common Errors in Exam

Error	Reason
ClassNotFoundException	Wrong class name
Output folder exists	Delete old output folder
Compilation error	Missing Hadoop classpath

Delete Old Output Folder

Before rerunning:

```
rm -r output
```

Very common viva point.

2-Minute Viva Explanation

"In this practical, we implemented the WordCount application using Hadoop MapReduce framework in Java. The input text file is processed by the Mapper class, which splits lines into words and emits key-value pairs in the form word and 1. During shuffle and sort, Hadoop groups identical words together. The Reducer class then sums the counts for each word and produces final word frequencies. We compiled the Java program, created a JAR file, and executed it using Hadoop in local standalone mode on Ubuntu."

Hadoop MapReduce Practical

Log File Analysis using MapReduce

This is another very common LP2/DSBDA Hadoop practical.

Aim

To design a distributed application using:

- Hadoop MapReduce
- Java

to process:

system log file

and count occurrences of:

- ERROR
- INFO
- WARN

messages.

What is a Log File?

A log file stores:

- system events
- errors
- warnings
- activity information

Example:

```
INFO User logged in  
ERROR Database failed  
WARN Memory low  
INFO File uploaded  
ERROR Network issue
```

Goal of Application

We want to count:

- how many ERROR messages
- how many INFO messages
- how many WARN messages

using:

MapReduce

Why Use Hadoop for Log Files?

Because log files can be:

- very large
- generated continuously

Hadoop helps:

- process logs in parallel
- improve speed

MapReduce Logic

Mapper Phase

Reads each line.

Extracts:

INFO / ERROR / WARN

Outputs:

ERROR 1

INFO 1

WARN 1

Reducer Phase

Adds counts for same key.

Example:

ERROR 5

INFO 10

WARN 2

Sample Input File

Create:

log.txt

Content:

INFO User login

ERROR Database failed

WARN Memory low

INFO File uploaded

ERROR Connection timeout

INFO Logout successful

Java MapReduce Code

File:

LogAnalysis.java

Complete Java Code

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class LogAnalysis {

    // Mapper Class
    public static class LogMapper
        extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);

        private Text logType = new Text();

        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();

            if (line.contains("ERROR")) {

                logType.set("ERROR");

                context.write(logType, one);

            } else if (line.contains("INFO")) {

                logType.set("INFO");
```

```

        context.write(logType, one);

    } else if (line.contains("WARN")) {

        logType.set("WARN");

        context.write(logType, one);
    }
}

// Reducer Class
public static class LogReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key,
                       Iterable<IntWritable> values,
                       Context context)
        throws IOException, InterruptedException {

        int sum = 0;

        for (IntWritable val : values) {

            sum += val.get();
        }

        context.write(key, new IntWritable(sum));
    }
}

// Main Method
public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(conf, "Log Analysis");

    job.setJarByClass(LogAnalysis.class);

    job.setMapperClass(LogMapper.class);

    job.setReducerClass(LogReducer.class);

    job.setOutputKeyClass(Text.class);

    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job,
        new Path(args[0]));

    FileOutputFormat.setOutputPath(job,

```



```
        new Path(args[1]));  
  
        System.exit(job.waitForCompletion(true)  
            ? 0 : 1);  
    }  
}
```

Explanation of Important Parts

```
line.contains("ERROR")
```

Checks whether line contains:

ERROR

```
context.write(logType, one)
```

Outputs:

(ERROR,1)

Reducer

Adds all counts for same log type.

Expected Mapper Output

```
INFO 1  
ERROR 1  
WARN 1  
INFO 1  
ERROR 1  
INFO 1
```

Final Reducer Output

ERROR 2

INFO 3

WARN 1

Compilation Commands in Ubuntu

Compile

```
javac -classpath `hadoop classpath` -d log_classes LogAnalysis.java
```

Create JAR

```
jar -cvf loganalysis.jar -C log_classes/ .
```

Run Hadoop Job

```
hadoop jar loganalysis.jar LogAnalysis log.txt output
```

View Output

```
cat output/part-r-00000
```

Important Theory

What is Distributed Processing?

Processing data across multiple systems/nodes simultaneously.

Why Hadoop Suitable for Logs?

Because logs are:

- huge
 - continuously generated
-

What is Mapper?

Processes input data.

What is Reducer?

Aggregates results.

What is Key-Value Pair?

Example:

ERROR → 1

What is Shuffle and Sort?

Intermediate phase where Hadoop:

- groups same keys together
-

Complete Flow

Log File



Mapper



(ERROR,1)

(INFO,1)

(WARN,1)



Shuffle & Sort



Reducer

Important Viva Questions

Basic Questions

What is log analysis?

Processing system logs to extract useful information.

Why Hadoop used for log analysis?

Handles very large log files efficiently.

What is Mapper role?

Extract log type and emit key-value pairs.

What is Reducer role?

Count occurrences of each log type.

Intermediate Questions

What does contains() do?

Checks substring presence.

What is key in this program?

ERROR / INFO / WARN

What is value in this program?

Why IntWritable used?

Hadoop integer datatype.

Advanced Questions

What happens between mapper and reducer?

Shuffle and sort phase.

Can Hadoop process TBs of logs?

Yes.

Difference between standalone and distributed mode?

Standalone	Distributed
Single machine	Multiple nodes

External May Ask Modifications

Count IP Addresses

Extract IP instead of ERROR/INFO.

Find Only ERROR Logs

Remove INFO/WARN conditions.

Convert Logs to Lowercase

```
line = line.toLowerCase();
```

Count Total Requests Per User

Extract username as key.

Common Exam Errors

Error	Reason
Output folder exists	Delete old output folder
Compilation failure	Missing Hadoop classpath
Wrong class name	Jar execution error

Delete Old Output Folder

```
rm -r output
```

2-Minute Viva Explanation

"In this practical, we designed a distributed log analysis application using Hadoop MapReduce in Java. The mapper reads each log entry and checks whether it contains INFO, ERROR, or WARN. It emits key-value pairs like ERROR and 1. Hadoop performs shuffle and sort to group identical keys. The reducer then sums the counts for each log type and produces final statistics. This application demonstrates distributed processing of large log files using the MapReduce framework."